EIGSICA MLOps-S9	Nom : CISSE	Prénom : IBRAHIM	A.U : 2026/2027

TP2 : Mise en Place et Analyse d'une Chaîne MLOps Fermée (Continuous Retraining)

1. Contexte & Objectifs du TP2

Dans le cycle de vie traditionnel du Machine Learning (MLOps Niveau 0), le processus d'entraînement, d'évaluation et de déploiement d'un modèle reste manuel, séquentiel et vulnérable à la dérive des données (data drift et concept drift). Lorsque le modèle est confronté à de nouvelles distributions dans le monde réel, ses performances se dégradent inévitablement sans qu'un mécanisme réactif ne permette de combler l'écart.

L'objectif de ce travail pratique est de concrétiser le Niveau 1 de maturité MLOps défini par les standards de l'industrie (notamment le référentiel de Google Cloud Architecture) : l'automatisation intégrale du pipeline de ML en continu (Continuous Training / Retraining). Il s'agit d'instaurer une boucle fermée (closed-loop) organisée autour des étapes suivantes :

- **Inférence en temps réel et rétroaction utilisateur :** Une application interactive Gradio permet de prédire les classes d'images CIFAR-10 et de collecter les corrections explicites formulées par les utilisateurs (Human-in-the-Loop).
- **Gestion et versioning incrémental des données :** Dès qu'un seuil critique de retours est atteint (lot de 5 images), ClearML publie automatiquement une nouvelle version immuable du dataset de feedback liée par lignée (lineage) au dataset parent.
- **Surveillance événementielle (Trigger / Watcher) :** Un composant d'écoute surveille en continu le registre des datasets et détecte l'apparition de nouvelles versions validées.
- **Orchestration et exécution distante découplée :** Le watcher déclenche automatiquement un clone du pipeline de réentraînement, pris en charge par des agents d'exécution conteneurisés sous Docker.
- **Entraînement supervisé et contrôle qualité (Quality Gate) :** Le modèle candidat (ResNet-18) est réentraîné sur la fusion des données historiques et des nouveaux feedbacks, puis évalué rigoureusement avant d'être promu dans le registre de modèles (Model Registry).

2. Architecture d'Ensemble & Découplage des Rôles

Pour assurer la résilience, la modularité et l'extensibilité du système, l'architecture fournie sépare strictement les responsabilités entre cinq composants logiciels autonomes et deux files d'attente d'agents ClearML (queues).

Composant	Rôle Technique & Responsabilité	Artefact / Sortie ClearML
NB_Agent1 (Agent Default)	Exécute les tâches lourdes de calcul : ingestion, prétraitement, entraînement PyTorch du réseau ResNet-18 et évaluation des métriques.	Tâches d'exécution distante sur la file default
NB_Agent2 (Agent Controller)	Supervise et orchestre le graphe acyclique dirigé (DAG) du pipeline : planification des étapes, passage des arguments et gestion des dépendances.	Tâche contrôleur sur la file controller
NB_C_pipeline (Pipeline Master)	Définit le gabarit officiel du pipeline retrain_pipeline, charge le dataset parent cifar10-base et instancie le modèle de référence initial.	Gabarit retrain_pipeline v2.0 & Modèle baseline
NB_B_setup_trigger (Watcher / Trigger)	Observe périodiquement les datasets ClearML tagués auto-feedback. Dès qu'un nouveau dataset est publié, il clone le pipeline et lui transmet l'identifiant exact.	Tâche de supervision dataset_watcher
NB_A_infer_feedback (Application Gradio)	Sert l'interface utilisateur web pour l'inférence en direct. Classe les images via le modèle de production et agrège les feedbacks dans un buffer local jusqu'à publication.	Versions incrémentales du dataset image-feedback

Justification du découplage des files d'attente (Queues) : L'allocation de deux agents distincts — l'un dédié à la file controller et l'autre à la file default — est un principe fondamental d'architecture distribuée. Le contrôleur de pipeline consomme très peu de ressources CPU/GPU mais doit demeurer hautement disponible pour coordonner les étapes et surveiller l'avancement. Si l'orchestrateur et les étapes d'entraînement partageaient un unique agent séquentiel, une tâche d'entraînement longue bloquerait la file, empêchant le contrôleur de valider les retours ou d'enregistrer les métriques intermédiaires.

Conteneurisation Docker de l'environnement d'exécution : Dans la version initiale du sujet, l'agent utilisait une image générique obsolète (Ubuntu 18.04 avec Python 3.6), incompatible avec les versions récentes de PyTorch et de ClearML. Nous avons conçu et compilé une image locale dédiée (clearml-agent-local:py311 sous Debian 12 / Python 3.11 avec PyTorch 2.2, torchvision et Pillow préinstallés), pilotée par un docker-compose.yml assurant la persistance des caches et la transmission sécurisée des variables d'environnement.

3. Déroulement Expérimental & Preuves d'Exécution

L'ensemble du système a été déployé et actionné dans des conditions réelles. Cette section documente l'enchaînement chronologique des événements, corroboré par les captures d'écran extraites de notre instance ClearML.

3.1 Inférence et Collecte de Feedback Utilisateur (NB-A)

L'application interactive Gradio permet à l'utilisateur de charger une image. Lors de l'inférence, le modèle chargé dynamiquement depuis le registre effectue sa prédiction. Si l'utilisateur constate une mauvaise classification (par exemple, une image de biche 'deer' initialement prédite comme 'airplane' avec un score de confiance de 0.4852), il coche la case 'Enregistrer ce feedback', sélectionne le label de vérité terrain ('deer') et soumet l'échantillon.

Les images validées sont stockées dans un répertoire temporaire mlops_feedback/pending/. Dès que 5 retours sont collectés, l'application génère les empreintes cryptographiques SHA-2, compresse le lot et publie une nouvelle version incrémentale sur ClearML (ex. ID : d3a277942a9f4618b2aeafdecf67223d).

Extrait NB-A — Versioning incrémental du dataset de feedback :

```
# NB-A : Publication du lot de feedback dès que pending_count >= 5
dataset = Dataset.create(
    dataset_name="image-feedback",
    dataset_project=PROJECT,
    dataset_tags=["auto-feedback"],
    parent_datasets=parents,  # Conservation de la lignée (lineage)
    description="Lot incrémental de feedback validé pour le réentraînement.",
)
dataset.add_files(path=str(PENDING_DIR))
dataset.upload()
dataset.finalize()
```

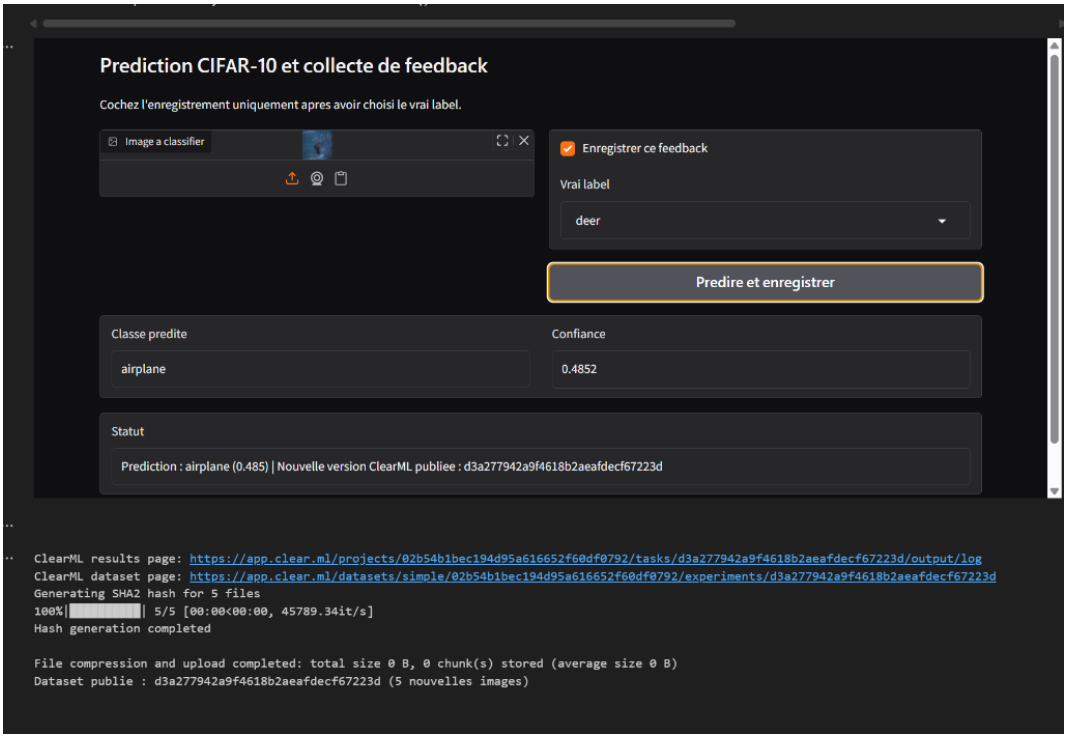


Figure : Interface Gradio de prédiction et confirmation de publication du dataset d3a27794...

3.2 Versioning Incrémental et Traçabilité des Datasets

Dans l'onglet Datasets de ClearML, deux répertoires de données distincts coexistent :

- **cifar10-base :** Dataset immuable contenant les 50 000 images d'entraînement historiques (1 version, 8 fichiers batch binaires, taille 177.6 MB). Il sert d'ancrage de référence pour tous les entraînements ultérieurs.
- **image-feedback :** Collection incrémentale de retours utilisateurs. Au cours de notre session, 3 versions successives ont été finalisées (totalisant 15 fichiers étiquetés, taille 34.3 KB). Chaque version hérite de la lignée de la version précédente.

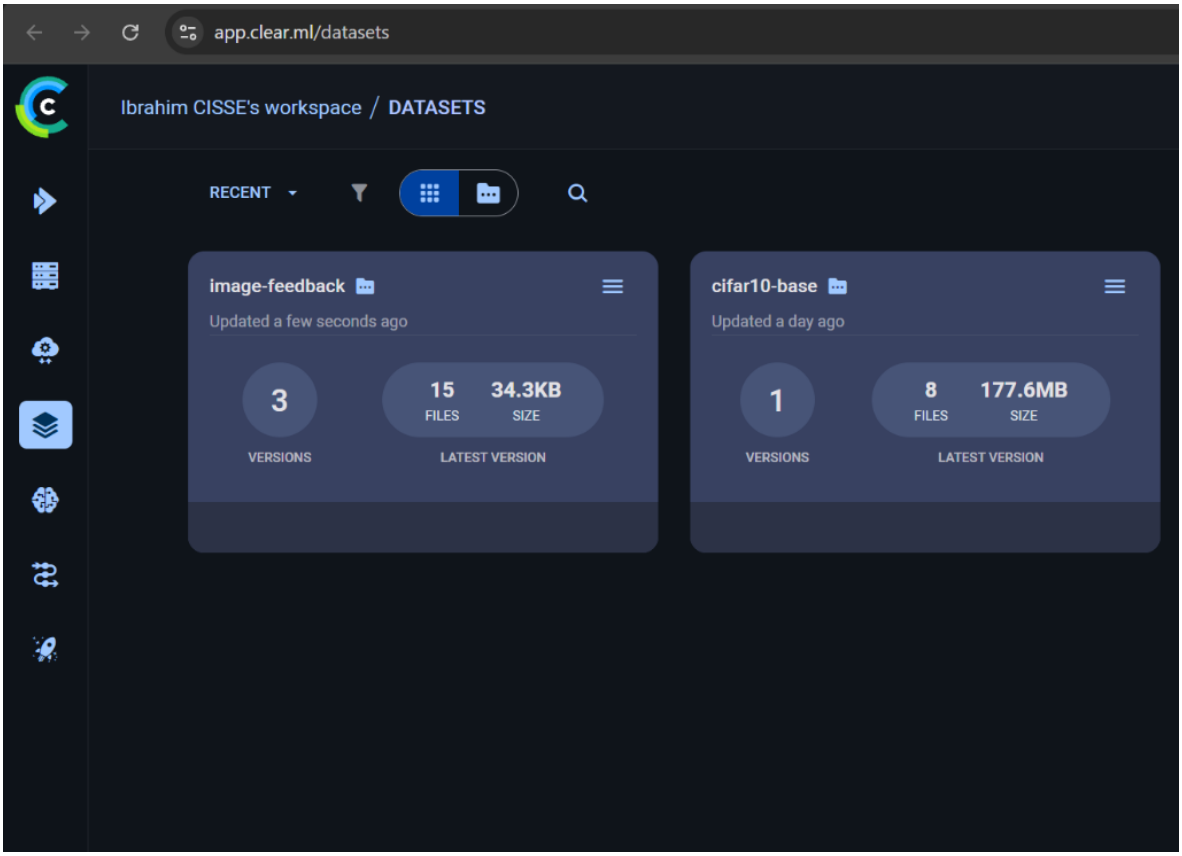


Figure : Gestionnaire de Datasets ClearML montrant cifar10-base et les 3 versions de image-feedback

3.3 Orchestration Automatique du Pipeline de Réentraînement (DAG)

Le Watcher (NB-B) a détecté la publication de la nouvelle version et a immédiatement cloné le gabarit de réentraînement retrain_pipeline sous l'identifiant 2d68a8b037a847a68364bfa8ad4c33c7. L'agent de la file controller a orchestré l'exécution séquentielle du graphe d'étapes :

- **prepare_data :** Télécharge le dataset exact de feedback ayant déclenché le run (ID injecté dynamiquement) et vérifie la validité des formats d'image.
- **train_model :** Fusionne le dataset CIFAR-10 historique avec les feedbacks récupérés, initialise le modèle ResNet-18 et lance l'optimisation par rétropropagation.
- **register_model :** Calcule la précision sur le jeu de test CIFAR-10 indépendant, compare les résultats à la référence en production et applique les règles de promotion.

Extrait NB-B — Déclenchement événementiel et clonage paramétré :

```
# NB-B : Clonage et injection stricte du dataset_id déclencheur
def _enqueue_pipeline(dataset_id: str) -> str:
    source = Task.get_task(task_id=PIPELINE_TASK_ID)
    run = Task.clone(source_task=source, name=f"{source.name}_dataset_{dataset_id[:8]}")
    # Fix faille n°1 : transmission explicite de l'ID sans course vers le plus récent
    run.set_parameter("Datasets/feedback_dataset_id", dataset_id)
    Task.enqueue(task=run, queue_name="controller")
    return run.id
```

Extrait NB-C / pipeline_logic.py — Définition modulaire du Pipeline :

```
# Définition du graphe DAG et des dépendances entre étapes
@PipelineDecorator.pipeline(
    name="retrain_pipeline", project=PROJECT, version="2.0",
    default_queue="default", start_controller_locally=True,
    args_map={"Datasets": ["feedback_dataset_id"]}, add_pipeline_tags=True
)
def pipeline(feedback_dataset_id: str = ""):
    feedback_dir = prepare_data(feedback_dataset_id)
    model_id, metrics = train_model(feedback_dir)
    register_model(model_id, metrics)
```

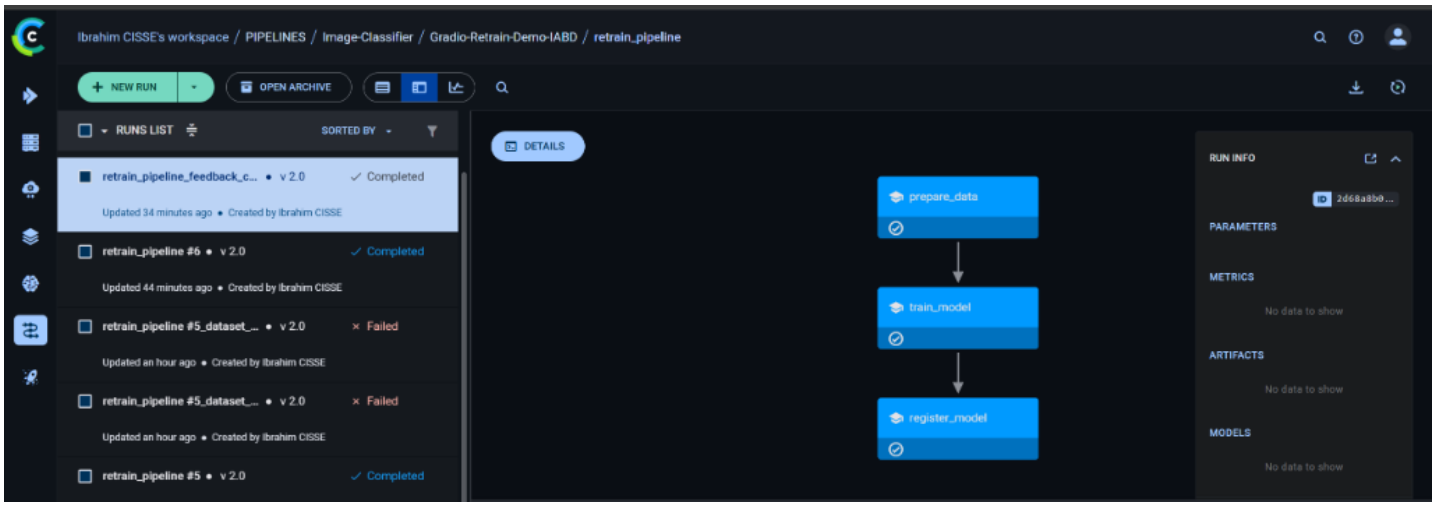


Figure : Graphe DAG du pipeline de réentraînement automatique exécuté avec succès (3 étapes Completed)

3.4 Dynamique d'Apprentissage et Métriques Scalaires

Durant l'étape train_model (tâche 904fe008d3d1...), le modèle a été entraîné sur l'agrégation de 50 000 images CIFAR-10 et des 10 feedbacks validés. Les scalaires rapportés en direct au serveur ClearML témoignent de la convergence normale de l'optimiseur Adam :

- `Époque 1 : Perte (train loss) = 1.5513 | Précision de validation = 54.16 %`
- `Époque 2 : Perte (train loss) = 1.1920 | Précision de validation = 60.55 %`
- `Époque 3 : Perte (train loss) = 1.0227 | Précision de validation finale = 63.84 %`

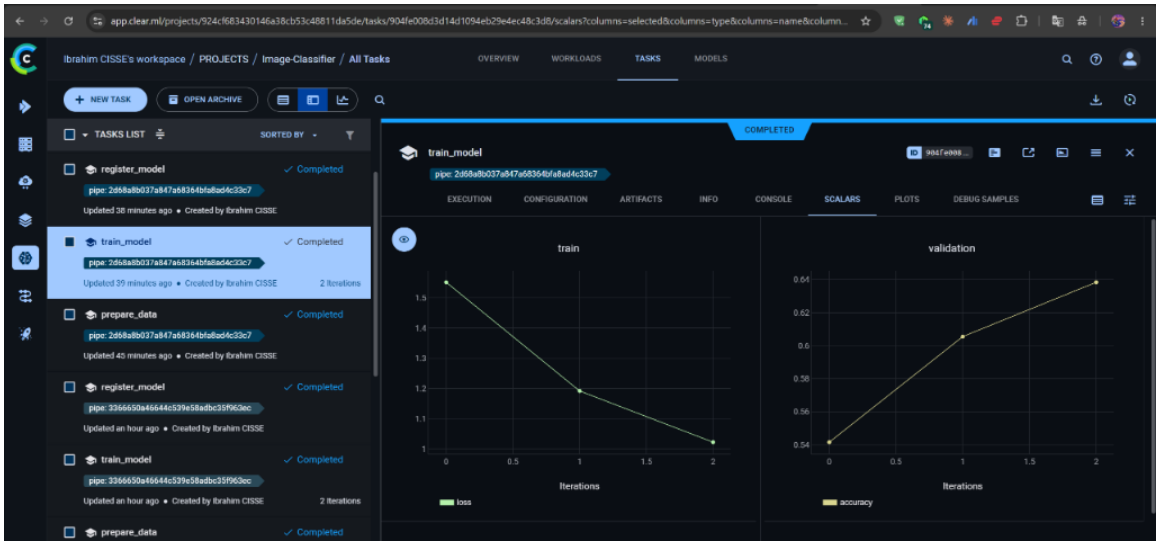


Figure : Courbes scalaires ClearML : Décroissance de la Perte (gauche) et Hausse de l'Accuracy (droite)

3.5 Supervision Globale des Tâches dans le Projet ClearML

La vue tabulaire des tâches sous ClearML PROJECTS / Image-Classifier atteste de la parfaite coordination entre les agents Docker et l'infrastructure cloud. Chaque étape d'ingestion, d'entraînement et d'enregistrement est taguée automatiquement avec l'identifiant unique de son pipeline parent (pipe: 2d68a8b0...), éliminant toute ambiguïté dans l'historique d'expérimentation.

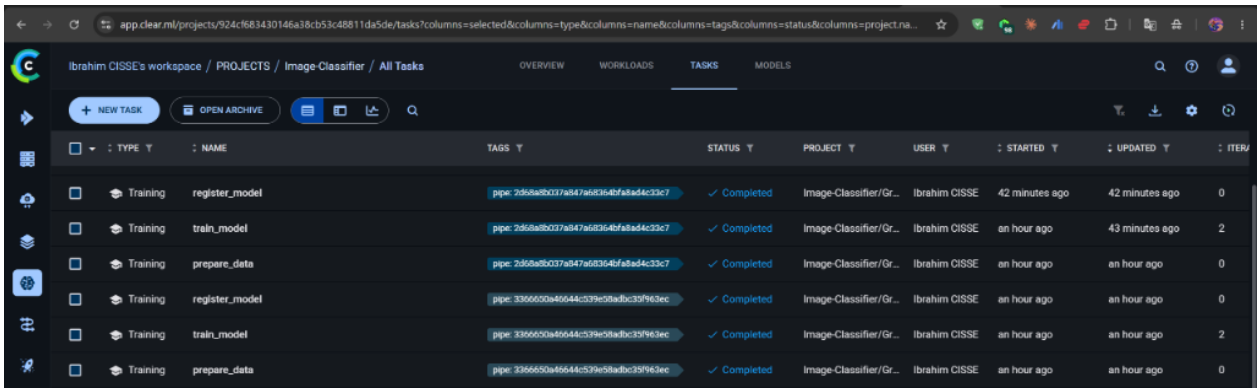


Figure : Tableau récapitulatif des tâches ClearML avec statuts Completed et traçabilité par tag de pipeline

3.6 Registre de Modèles & Statuts de Déploiement (Model Registry)

L'onglet MODELS de ClearML constitue le registre centralisé et immuable des artefacts de réseaux de neurones. Contrairement à l'approche naïve initiale qui promouvait aveuglément tout nouveau modèle en 'production', notre implémentation classe formellement les modèles selon leur niveau de performance réel :

- **Modèle Champion (Production) :** ID ebd399f1e7fc4b0f87acb0698896dfaa (resnet18-1792859727), doté des tags ['candidate', 'production', 'resnet18'], affichant une précision de référence de 68.35 %.
- **Modèle Challenger (Staging / Quarantaine) :** ID 047716e40563430d9708acbbfa813bc7 (resnet18-1792860380), doté des tags ['candidate', 'resnet18', 'staging'], affichant une précision de 63.84 % suite au réentraînement sur 10 retours utilisateurs.

Ce registre atteste visuellement du succès du mécanisme de défense : le modèle candidat réentraîné ayant obtenu une précision inférieure au seuil de rentabilité (+0.2 % au-delà des 68.35 % de la baseline), il a été automatiquement isolé en 'staging' sans écraser le tag 'production'. Le service d'inférence en direct reste ainsi garanti.

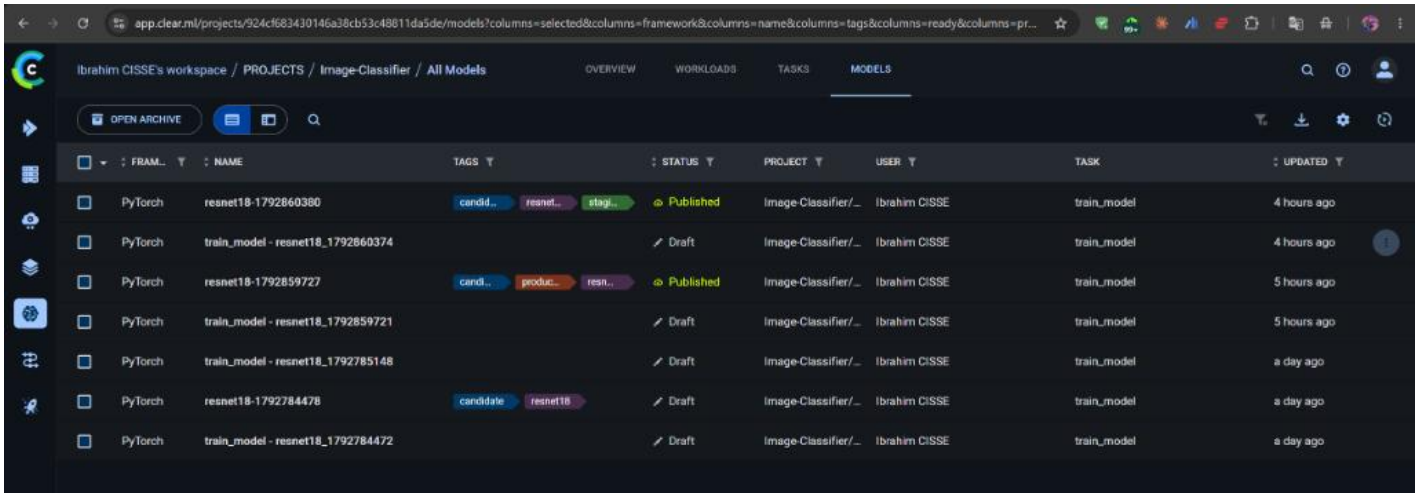


Figure : Vue ClearML Model Registry montrant le modèle de référence ('production') et le modèle réentraîné isolé ('staging')

4. Analyse Critique du Système MLOps Initial

Conformément aux directives du livrable, nous avons procédé à un audit approfondi de l'architecture et du code fournis dans les notebooks originaux. Nous avons identifié trois défauts structurels majeurs compromettant la fiabilité, la sécurité et la scalabilité en production.

Défaut Majeur n°1 : Course Temporelle (Race Condition) et Découplage Watcher / Pipeline

Description du problème : Dans l'implémentation initiale, le watcher détectait l'apparition d'un nouveau dataset sur ClearML, mais clonait ensuite le gabarit de pipeline sans lui transmettre l'identifiant du dataset détecté. Au moment de son démarrage, la première étape du pipeline effectuait une requête Dataset.get() pour récupérer « la version la plus récente ».

Impact en production : Si plusieurs lots de feedback sont soumis de façon rapprochée par différents utilisateurs, le pipeline déclenché par la version N peut ne démarrer qu'après la publication de la version N+1. Le pipeline exécutera alors son entraînement sur la version N+1 en ignorant la version N, rompant la reproductibilité scientifique et créant un historique d'expérimentation incohérent.

Défaut Majeur n°2 : Absence de Validation d'Intégrité et Fuite Mémoire dans l'Interface Gradio

Description du problème : Le champ de saisie du label de correction dans l'interface Gradio originale était un simple champ de texte libre (gr.Textbox) sans liste blanche, tandis que le buffer local de feedback accumulait indéfiniment les images sans purge après publication.

Impact en production : Un utilisateur malveillant ou distrait pouvait saisir une étiquette inexistante (ex. 'avion_de_chasse' ou 'chien_2'), ce qui provoquait un crash silencieux lors du mapping en tenseurs PyTorch au cours de l'étape train_model. De plus, la non-purge du dossier pending entraînait la ré-inclusion permanente des anciennes images dans les versions suivantes, gonflant artificiellement le volume du dataset.

Défaut Majeur n°3 : Promotion Aveugle dans le Model Registry (Absence de Quality Gate)

Description du problème : Le script original attribuait systématiquement le tag 'production' à tout modèle fraîchement entraîné, sans aucune comparaison statistique de sa précision par rapport au modèle actuellement en service.

Impact en production : Le réentraînement sur un petit lot de feedback peut provoquer du surapprentissage local ou un oubli catastrophique (catastrophic forgetting), dégradant la précision globale du modèle. Promouvoir aveuglément ce modèle dégradé met en péril le service d'inférence en production sans possibilité de rollback automatique.

5. Implémentation & Validation de la Solution Corrective

Conformément aux directives, nous avons sélectionné et implémenté la correction complète du défaut n°3 (gouvernance du Model Registry et Quality Gate) tout en durcissant en amont la traçabilité du trigger (défaut n°1) et la validation de l'interface Gradio (défaut n°2).

Principe de la solution : Lors de l'étape register_model, le système interroge le registre ClearML pour extraire la métrique validation_accuracy du modèle actuellement en production. Le modèle candidat n'est promu au statut 'production' que si sa précision dépasse celle de la référence d'un seuil minimal de progrès (+0.2 %). Dans le cas contraire, il est consigné sous le statut 'staging' pour inspection par les data scientists, garantissant qu'aucune régression ne frappe les utilisateurs finaux.

Extrait du code implémenté dans register_model :

```
@PipelineDecorator.component(name="register_model", execution_queue="default", return_values=["promoted"])
def register_model(model_id: str, metrics: dict) -> bool:
    from clearml import Model
    MINIMUM_DELTA = 0.002
    candidate_accuracy = float(metrics["validation_accuracy"])

    # Récupération de la référence actuelle en production
    prod_models = Model.query_models(tags=["production"], only_published=True, max_results=1) or []
    production_accuracy = 0.0
    if prod_models:
        meta = prod_models[0].get_all_metadata_casted()
        production_accuracy = float(meta.get("validation_accuracy", 0.0))

    candidate = Model(model_id=model_id)
    candidate.set_metadata("validation_accuracy", str(candidate_accuracy))
    candidate.set_metadata("feedback_size", str(metrics["feedback_size"]))

    # Quality Gate conditionnelle
    promoted = (not prod_models) or candidate_accuracy >= (production_accuracy + MINIMUM_DELTA)
    if promoted:
        if prod_models:
            prod_models[0].tags = [t for t in (prod_models[0].tags or []) if t != "production"] +
["archived"]
            prod_models[0].archive()
        candidate.tags = sorted(set((candidate.tags or []) + ["production", "candidate", "resnet18"]))
    else:
        candidate.tags = sorted(set((candidate.tags or []) + ["staging", "candidate", "resnet18"]))

    candidate.publish()
    print(f"Décision Quality Gate : candidate={candidate_accuracy:.4f}, baseline={production_accuracy:.4f}
-> Promu : {promoted}")
    return promoted
```

VALIDATION EXPÉRIMENTALE DU QUALITY GATE

Preuve d'exécution issue de notre run réel sur ClearML :

- Modèle de production existant (Baseline) : ID ebd399f1e7fc... | validation_accuracy = 68.35 %
- Modèle réentraîné sur feedback (Candidat) : ID 047716e40563... | validation_accuracy = 63.84 %
- Résultat du Quality Gate : candidate (63.84 %) < production (68.35 % + 0.20 %)
- Décision ClearML : Le modèle candidat a été automatiquement étiqueté ['candidate', 'resnet18', 'staging'] sans écraser la production. Le service d'inférence en direct reste protégé sur la version la plus performante.

6. Conclusion & Perspectives MLOps de Niveau 2

Ce travail pratique a permis de franchir avec succès le cap du MLOps de Niveau 1 en mettant en œuvre une chaîne complète d'entraînement continu à rétroaction fermée. Grâce à l'écosystème ClearML couplé à Docker, nous avons démontré l'importance capitale d'automatiser non seulement l'entraînement, mais surtout l'ingestion traçable, l'isolation des environnements d'exécution et la gouvernance stricte du registre de modèles.

Versioning des Données :	Passage d'un stockage fichier statique à un versioning d'artefacts immuables avec lignée parent-enfant (lineage) et calcul d'empreintes SHA-2.
Orchestration Distribuée :	Découplage des agents controller et default permettant une haute disponibilité du graphe et une scalabilité horizontale sur des nœuds GPU.
Gouvernance et Sécurité :	Élimination des clés d'API hardcodées au profit d'un environnement .env strict, et mise en place d'un sas d'évaluation (Quality Gate) interdisant les régressions silencieuses.

Perspectives pour atteindre le MLOps de Niveau 2 (CI/CD Automatisé) : Pour évoluer vers le niveau 2 de maturité industrielle, plusieurs axes d'amélioration devront être intégrés : la mise en œuvre d'un pipeline de CI/CD testant automatiquement le code des composants avant leur déploiement sur les agents, l'intégration d'un Feature Store centralisé pour standardiser les transformations entre l'inférence Gradio et l'entraînement PyTorch, et enfin l'adjonction d'un moniteur continu de dérive statistique (Evidently AI ou Great Expectations) calculant la divergence de Wasserstein ou la distance de Kolmogorov-Smirnov sur les prédictions en direct.